



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Agile Development Process and Devops Lab – ISWE406P

Final Assessment Test

Faculty: Dr. Bala Ganesh N

Slot: L3 + L4

Name: Yukesh R M

Register Number: 23MIS0536

SET – 1

Online Food Ordering System

Problem Statement:

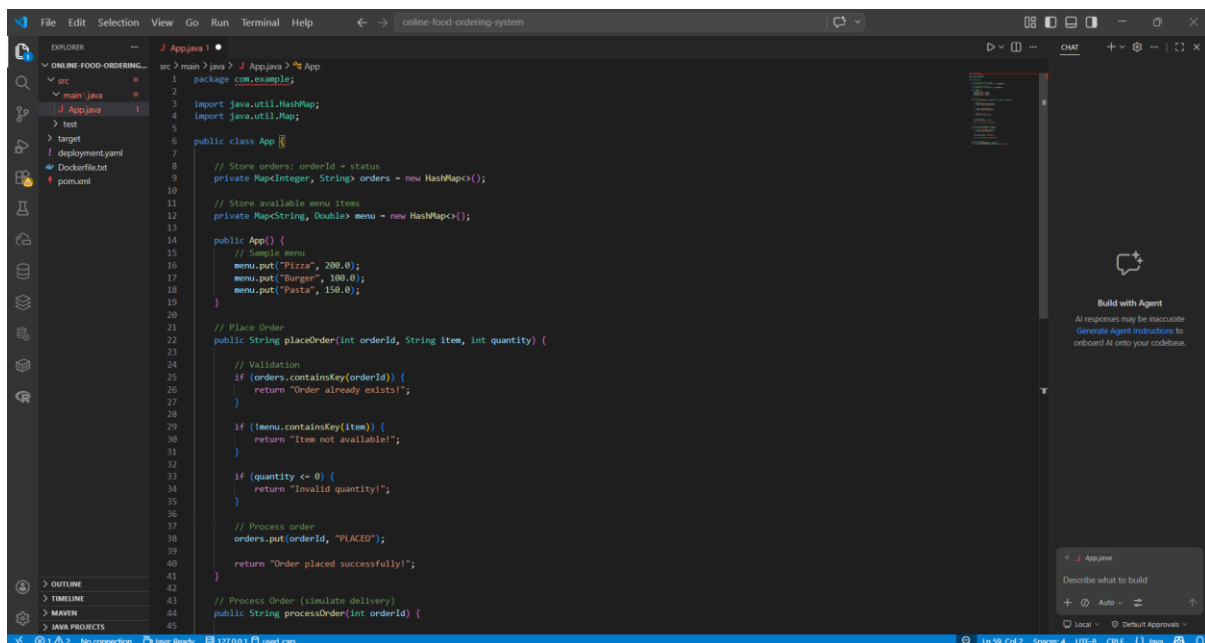
Ensure food orders are placed and processed correctly without errors.

Tasks:

1. Develop order placement and processing logic
2. Write JUnit tests for order success/failure scenarios
3. Implement CI/CD pipeline with automated testing
 - Set up version control using GitHub
 - Configure CI pipeline(e.g., Jenkins/GitHub Actions) to trigger on code commits
 - Automate build process using Maven
 - Integrate automated unit tests in pipeline
 - Configure CD pipeline for automatic deployment to staging/production
4. Deploy services on Docker and Kubernetes
 - Containerize application using Docker(write Dockerfile)
 - Build and push Docker image to container registry (Docker Hub)
 - Create Kubernetes deployment and service YAML files

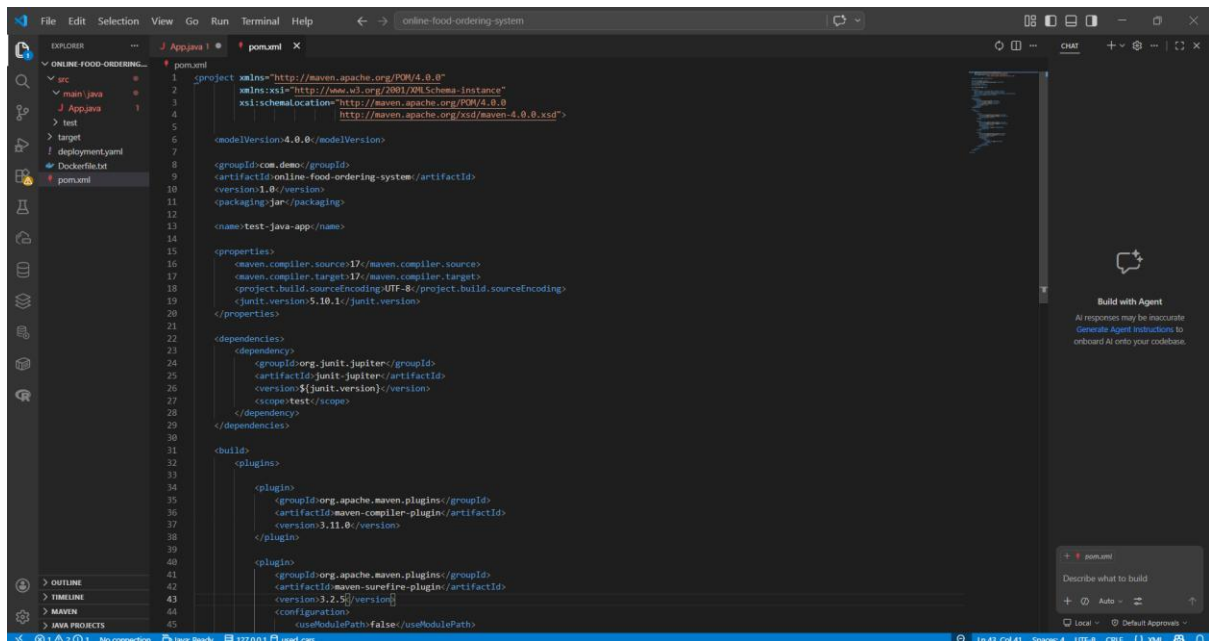
Step 1: Develop order placement and processing logic

Create App.java



```
1 package com.example;
2
3 import java.util.HashMap;
4 import java.util.Map;
5
6 public class App {
7
8     // Store orders: orderId - status
9     private Map<Integer, String> orders = new HashMap<>();
10
11     // Store available menu items
12     private Map<String, Double> menu = new HashMap<>();
13
14     public App() {
15         // Sample menu
16         menu.put("Pizza", 200.0);
17         menu.put("Burger", 100.0);
18         menu.put("Pasta", 150.0);
19     }
20
21     // Place Order
22     public String placeOrder(int orderId, String item, int quantity) {
23
24         // Validation
25         if (orders.containsKey(orderId)) {
26             return "Order already exists!";
27         }
28
29         if (!menu.containsKey(item)) {
30             return "Item not available!";
31         }
32
33         if (quantity <= 0) {
34             return "Invalid quantity!";
35         }
36
37         // Process order
38         orders.put(orderId, "PLACED");
39         return "Order placed successfully!";
40     }
41
42     // Process Order (simulate delivery)
43     public String processOrder(int orderId) {
44
45     }
```

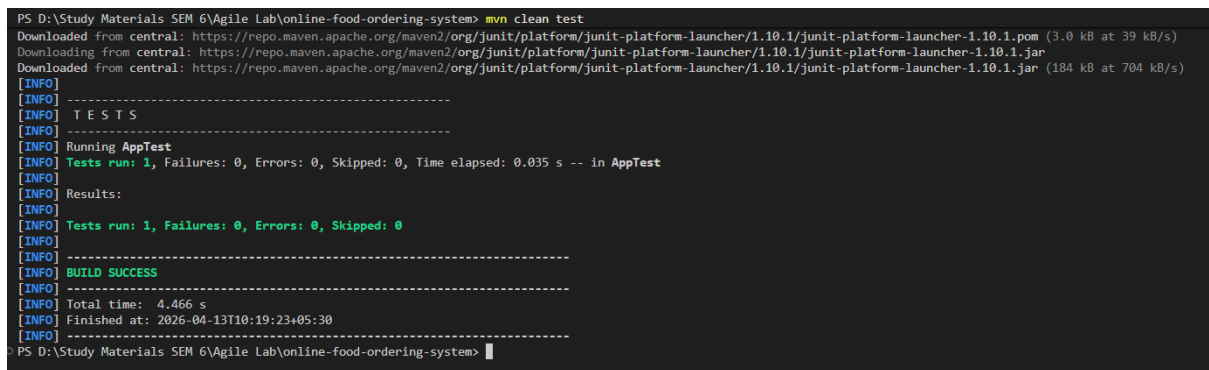
Create pom.xml



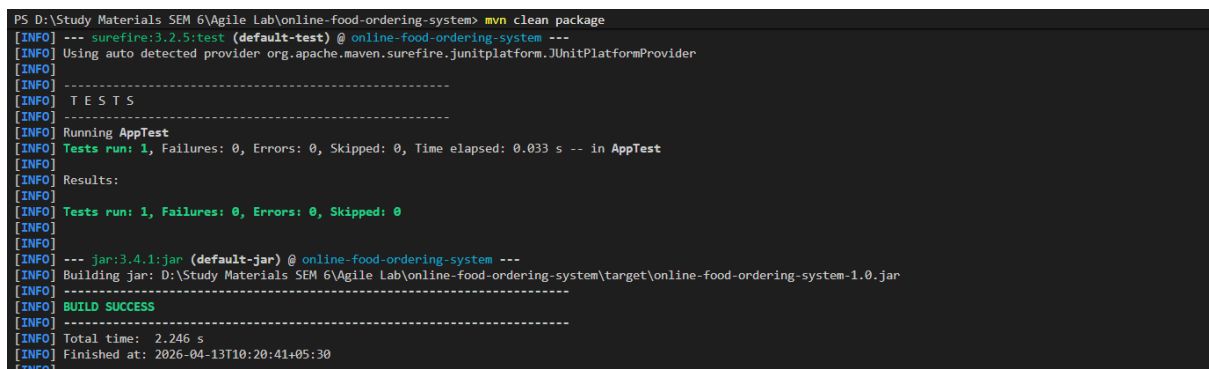
Build Locally using Maven (Test)

Inside project folder:

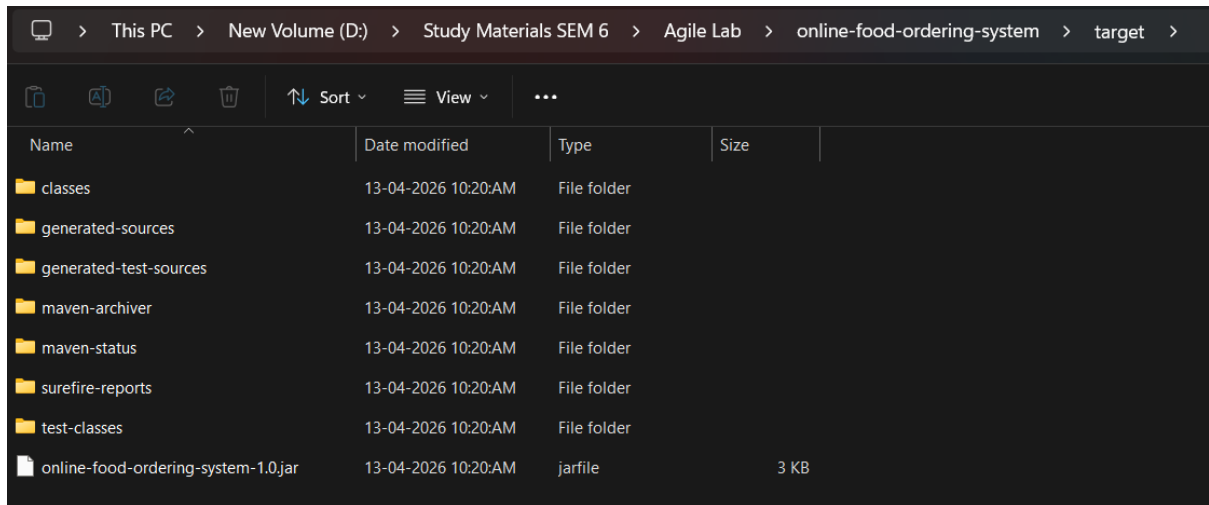
mvn clean test



mvn clean package

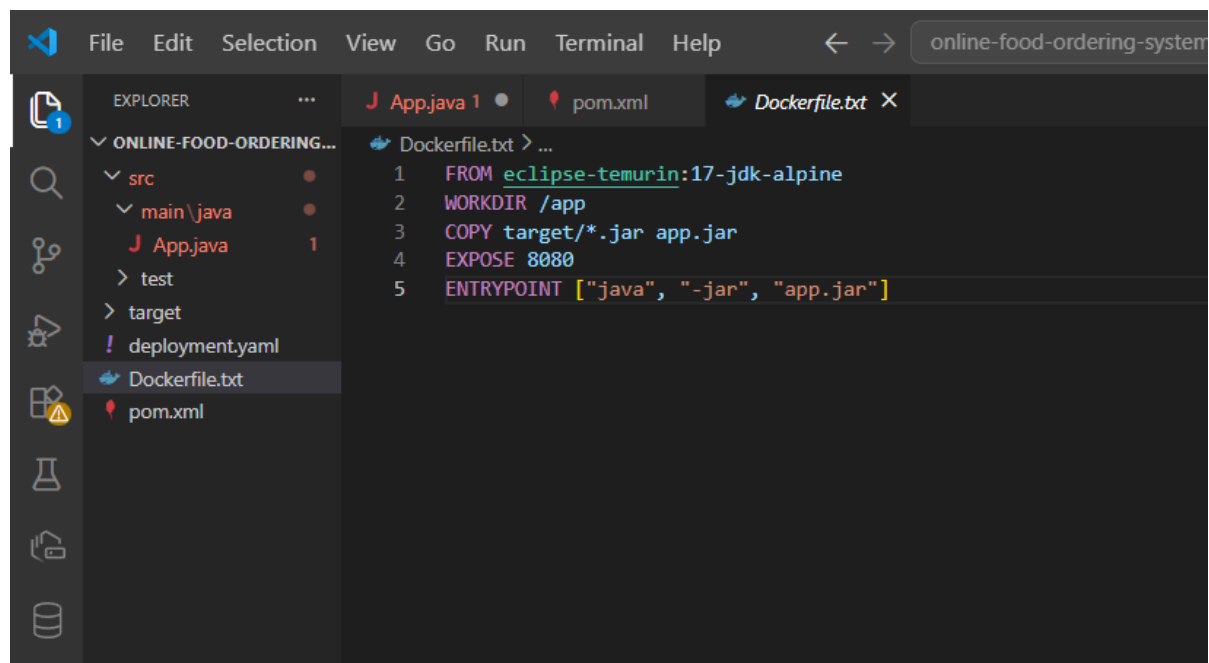


It generates: target/ online-food-ordering-system-1.0.jar



Name	Date modified	Type	Size
classes	13-04-2026 10:20:AM	File folder	
generated-sources	13-04-2026 10:20:AM	File folder	
generated-test-sources	13-04-2026 10:20:AM	File folder	
maven-archiver	13-04-2026 10:20:AM	File folder	
maven-status	13-04-2026 10:20:AM	File folder	
surefire-reports	13-04-2026 10:20:AM	File folder	
test-classes	13-04-2026 10:20:AM	File folder	
online-food-ordering-system-1.0.jar	13-04-2026 10:20:AM	jarfile	3 KB

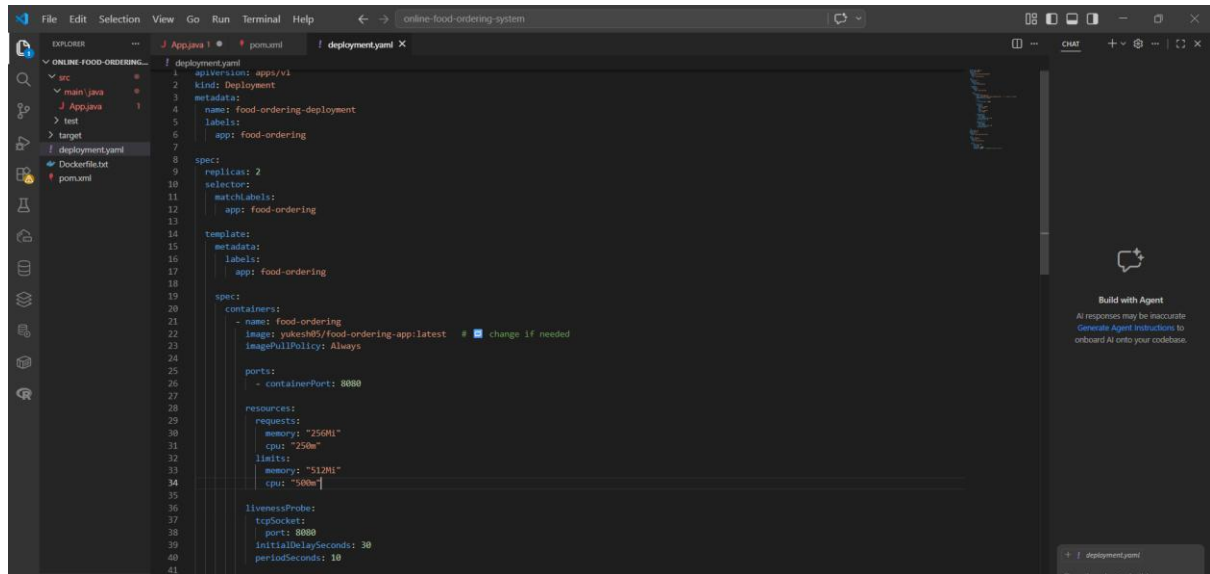
Create Dockerfile



```
1 FROM eclipse-temurin:17-jdk-alpine
2 WORKDIR /app
3 COPY target/*.jar app.jar
4 EXPOSE 8080
5 ENTRYPOINT ["java", "-jar", "app.jar"]
```

Create Kubernetes Deployment File

deployment.yaml

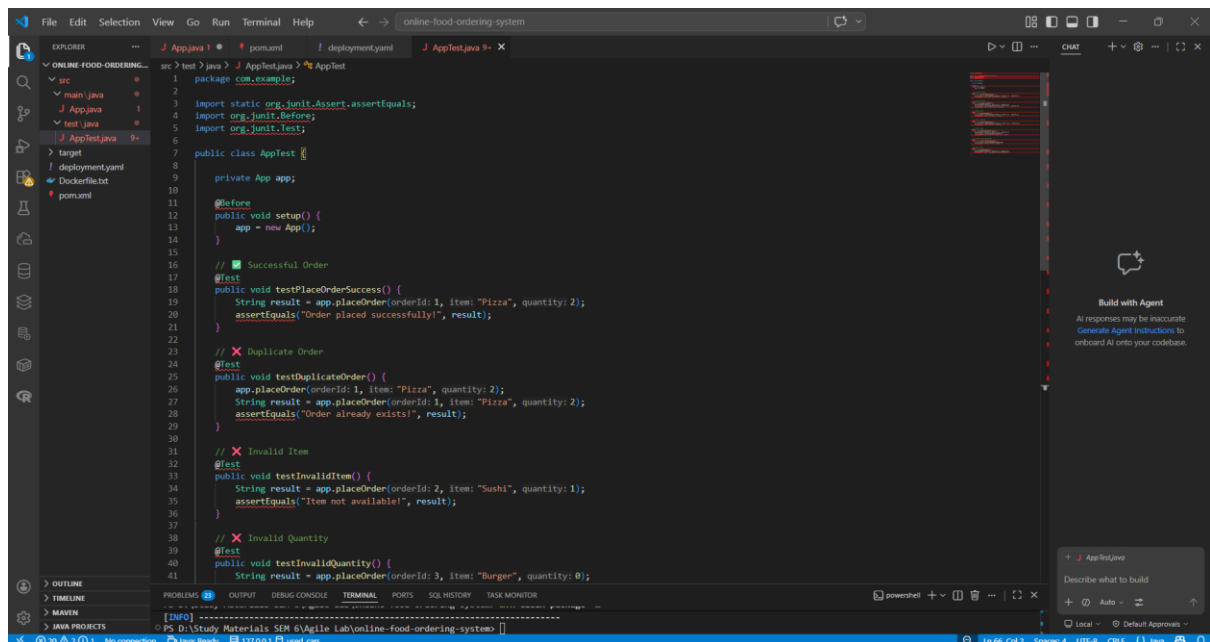


The screenshot shows the VS Code interface with the `deployment.yaml` file open in the Editor. The Explorer view on the left shows the project structure with `deployment.yaml` selected. The Editor view displays the following YAML content:

```
1 deployment.yaml
2 kind: Deployment
3 metadata:
4   name: food-ordering-deployment
5   labels:
6     app: food-ordering
7
8 spec:
9   replicas: 2
10  selector:
11    matchLabels:
12      app: food-ordering
13
14  template:
15    metadata:
16      labels:
17        app: food-ordering
18
19    spec:
20      containers:
21        - name: food-ordering
22          image: yuhesh05/food-ordering-app:latest # change if needed
23          imagePullPolicy: Always
24
25      ports:
26        - containerPort: 8080
27
28      resources:
29        requests:
30          memory: "250Mi"
31          cpu: "250m"
32        limits:
33          memory: "512Mi"
34          cpu: "500m"
35
36      livenessProbe:
37        tcpSocket:
38          port: 8080
39        initialDelaySeconds: 30
40        periodSeconds: 10
41
```

Step 2: Write JUnit tests for order success/failure scenarios

Create AppTest.java

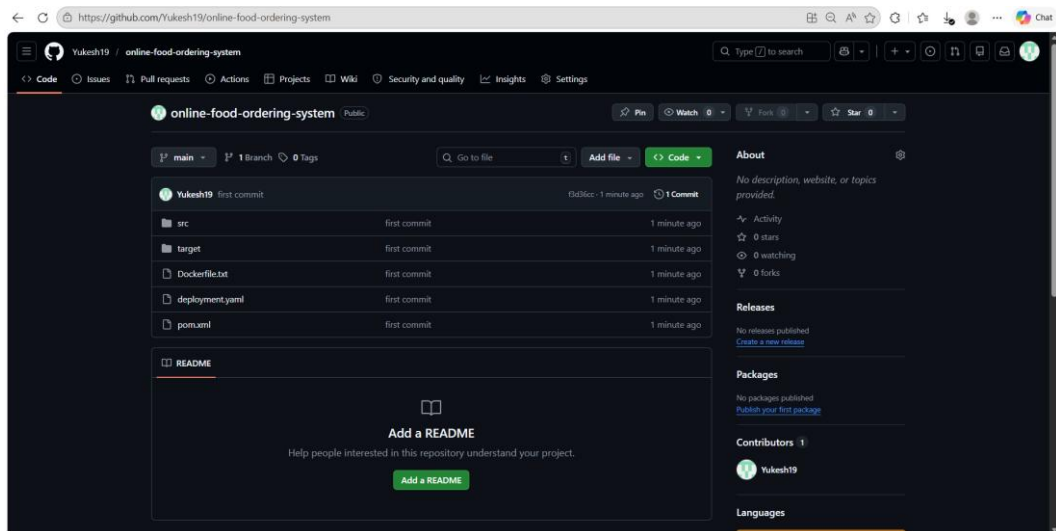


The screenshot shows the VS Code interface with the `AppTest.java` file open in the Editor. The Explorer view on the left shows the project structure with `AppTest.java` selected. The Editor view displays the following Java code:

```
1 package com.example;
2
3 import static org.junit.Assert.assertEquals;
4 import org.junit.Before;
5 import org.junit.Test;
6
7 public class AppTest {
8
9     private App app;
10
11     @Before
12     public void setUp() {
13         app = new App();
14     }
15
16     // Successful Order
17     @Test
18     public void testPlaceOrderSuccess() {
19         String result = app.placeOrder(1, item: "Pizza", quantity: 2);
20         assertEquals("Order placed successfully", result);
21     }
22
23     // Duplicate Order
24     @Test
25     public void testDuplicateOrder() {
26         app.placeOrder(1, item: "Pizza", quantity: 2);
27         String result = app.placeOrder(1, item: "Pizza", quantity: 2);
28         assertEquals("Order already exists!", result);
29     }
30
31     // Invalid Item
32     @Test
33     public void testInvalidItem() {
34         String result = app.placeOrder(2, item: "Sushi", quantity: 1);
35         assertEquals("Item not available!", result);
36     }
37
38     // Invalid Quantity
39     @Test
40     public void testInvalidQuantity() {
41         String result = app.placeOrder(3, item: "Burger", quantity: 0);
42     }
43 }
```

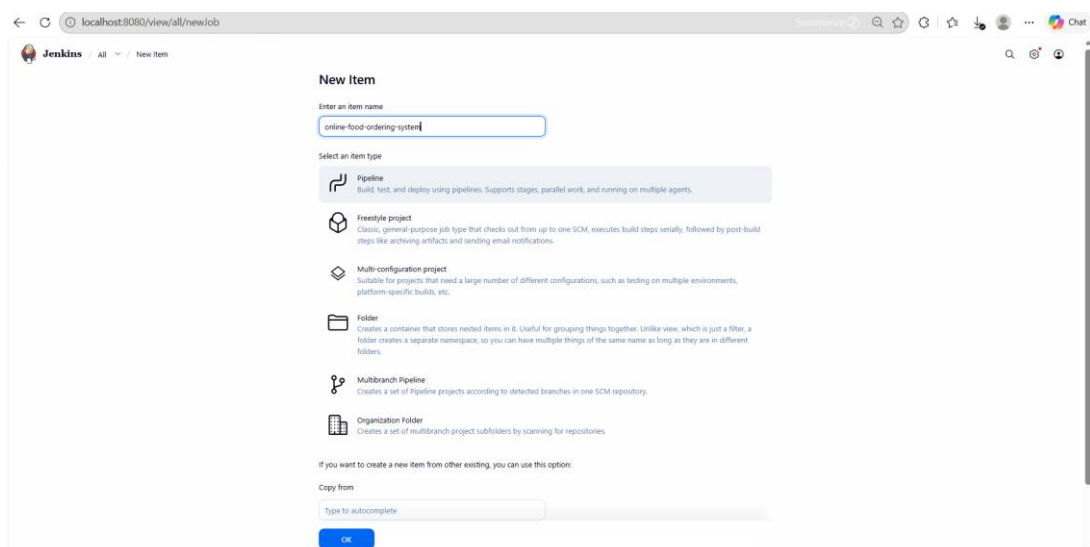
Push code to GitHub

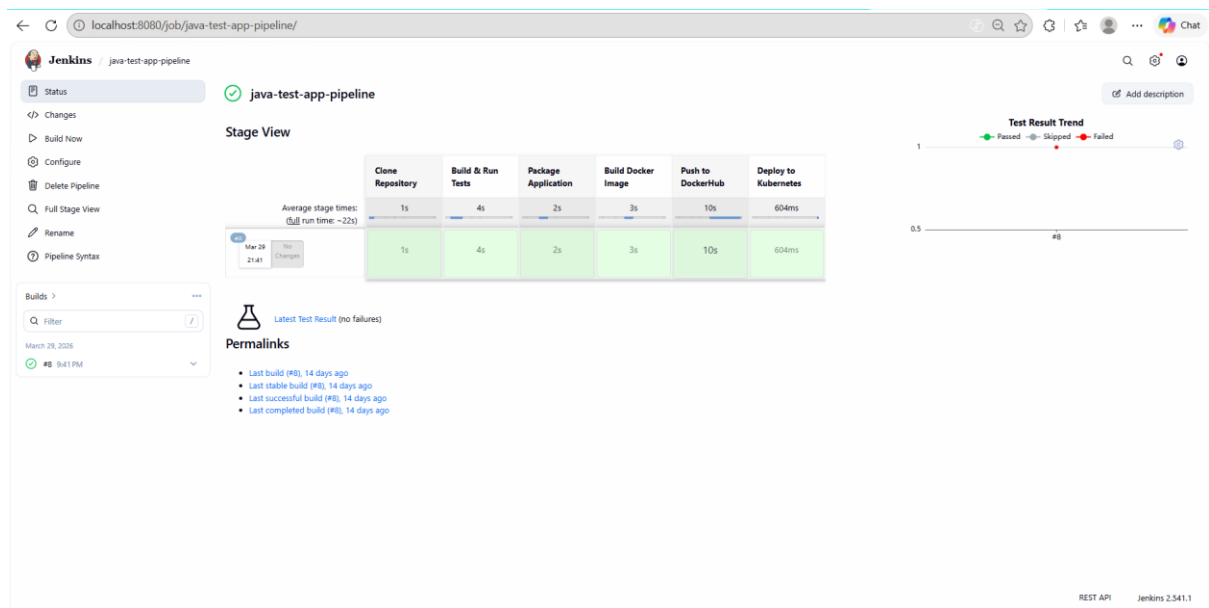
```
PS D:\Study Materials SEM 6\Agile Lab\online-food-ordering-system> git commit -m "first commit"
create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/inputFiles.lst
create mode 100644 target/online-food-ordering-system-1.0.jar
create mode 100644 target/surefire-reports/2026-04-13T10-20-40_799.dumpstream
create mode 100644 target/surefire-reports/AppTest.txt
create mode 100644 target/surefire-reports/TEST-AppTest.xml
create mode 100644 target/test-classes/AppTest.class
PS D:\Study Materials SEM 6\Agile Lab\online-food-ordering-system> git branch -M main
PS D:\Study Materials SEM 6\Agile Lab\online-food-ordering-system> git remote add origin https://github.com/Yukesh19/online-food-ordering-system.git
PS D:\Study Materials SEM 6\Agile Lab\online-food-ordering-system> git push -u origin main
Enumerating objects: 34, done.
Counting objects: 100% (34/34), done.
Delta compression using up to 12 threads
Compressing objects: 100% (22/22), done.
Writing objects: 100% (34/34), 8.32 KiB | 851.00 KiB/s, done.
Total 34 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Yukesh19/online-food-ordering-system.git
 * [new branch]    main -> main
branch 'main' set up to track 'origin/main'.
PS D:\Study Materials SEM 6\Agile Lab\online-food-ordering-system>
```



Step 3: Implement CI/CD pipeline with automated testing

Create Jenkins Pipeline Job

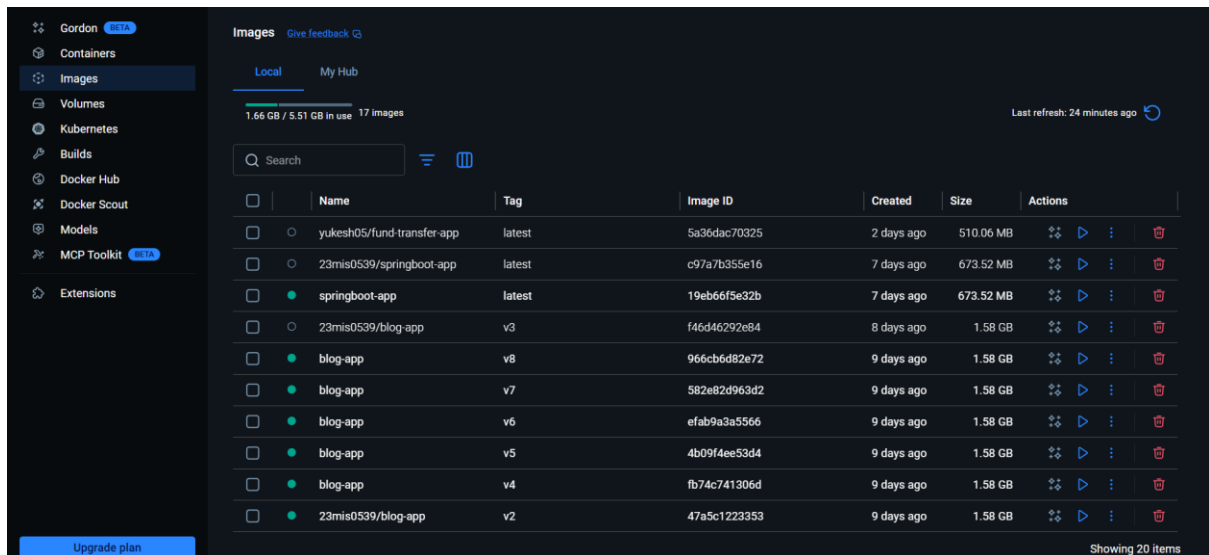


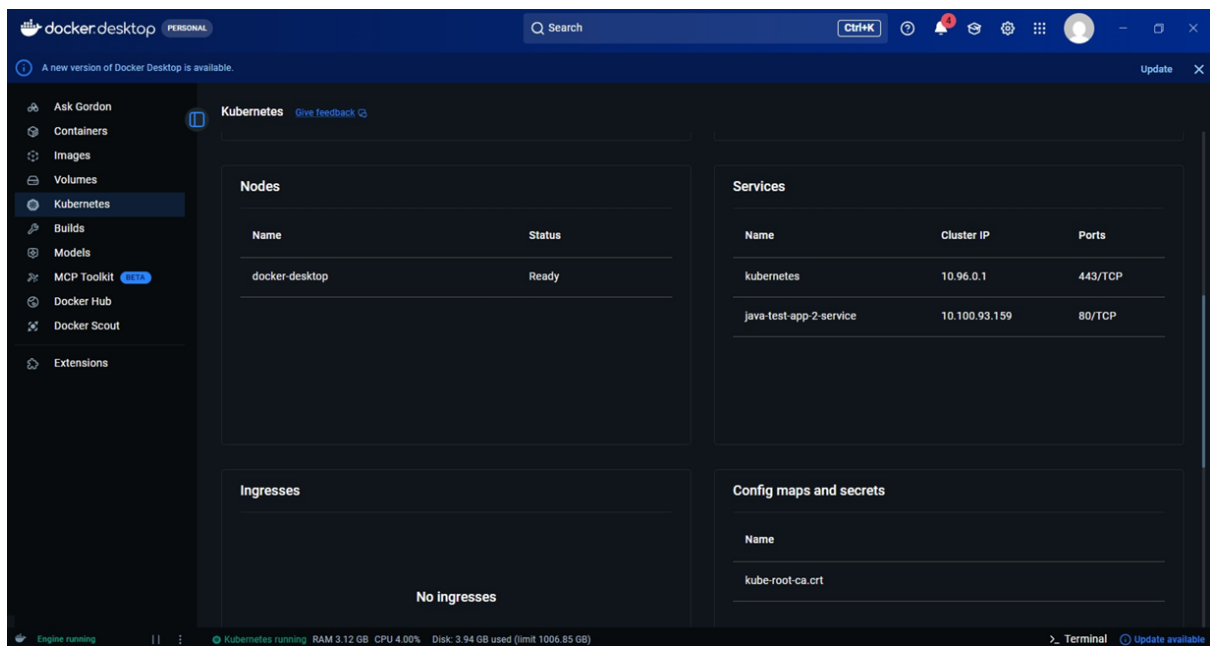


Step 4: Deploy micro service on Docker and Kubernetes

Verify Docker and Kubernetes Deployment

Docker Image





```
C:\Users\ryuke>kubectl get svc
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
java-test-app-service              NodePort    10.96.191.227 <none>         80:30007/TCP    5m29s
kubernetes                          ClusterIP   10.96.0.1     <none>         443/TCP          144m
```